

# LLM Context Compaction Proxy

## 技术文档

永不超限, 永不停歇 · Never Full, Never Stop

LLM Context Compaction Proxy Contributors

2026 年 8 月

## 目录

|                                               |           |
|-----------------------------------------------|-----------|
| <b>1 概述</b>                                   | <b>4</b>  |
| 1.1 工作原理                                      | 4         |
| 1.2 环境要求                                      | 4         |
| <b>2 架构设计</b>                                 | <b>4</b>  |
| 2.1 请求流程                                      | 4         |
| 2.2 模块划分                                      | 5         |
| <b>3 压缩技术栈</b>                                | <b>5</b>  |
| 3.1 V3 核心技术 (8 项)                             | 5         |
| 3.2 V4 增强技术 (10 项)                            | 6         |
| <b>4 核心算法解析</b>                               | <b>7</b>  |
| 4.1 CJK 感知的 Token 估算                          | 7         |
| 4.2 ARC 地址化引用 (Addressable Recall Compaction) | 7         |
| 4.3 PACMS 子模选择                                | 7         |
| 4.4 压缩安全验证                                    | 8         |
| <b>5 压缩流水线详解</b>                              | <b>8</b>  |
| 5.1 do_compaction 主流程                         | 8         |
| 5.2 增量压缩                                      | 9         |
| 5.3 双层压缩 (Dual-Layer)                         | 9         |
| <b>6 协议转换层</b>                                | <b>10</b> |
| 6.1 OpenAI → Anthropic 请求转换                   | 10        |
| 6.2 Anthropic → OpenAI 响应转换                   | 10        |
| 6.3 自动 Provider 探测                            | 10        |

|                                 |           |
|---------------------------------|-----------|
| <b>7 可靠性组件状态机</b>               | <b>10</b> |
| 7.1 熔断器 (CircuitBreaker)        | 10        |
| 7.2 抖动检测 (ThrashingDetector)    | 11        |
| 7.3 压缩缓存 (CompactionCache)      | 11        |
| 7.4 格式回退与重试                     | 11        |
| <b>8 会话存储架构</b>                 | <b>11</b> |
| 8.1 SessionStore(SQLite + FTS5) | 11        |
| 8.2 SemanticMemory(情节—语义双层记忆)   | 12        |
| 8.3 UserProfile(用户画像)           | 12        |
| <b>9 配置说明</b>                   | <b>12</b> |
| 9.1 核心配置                        | 12        |
| 9.2 压缩行为                        | 12        |
| 9.3 双引擎压缩                       | 13        |
| 9.4 安全                          | 13        |
| 9.5 会话与记忆                       | 13        |
| <b>10 接入指南</b>                  | <b>13</b> |
| 10.1 DeepSeek                   | 13        |
| 10.2 DeepSeek Harness           | 14        |
| 10.3 Hermes Agent               | 14        |
| 10.4 Claude Code                | 15        |
| 10.5 OpenClaw                   | 15        |
| <b>11 原有上下文系统屏蔽</b>             | <b>16</b> |
| 11.1 屏蔽原则                       | 16        |
| 11.2 OpenClaw                   | 16        |
| 11.3 Hermes Agent               | 17        |
| 11.4 DeepSeek Harness           | 17        |
| 11.5 Claude Code                | 17        |
| 11.6 屏蔽效果验证                     | 18        |
| <b>12 安全与可靠性</b>                | <b>18</b> |
| 12.1 认证                         | 18        |
| 12.2 密钥保护                       | 18        |
| 12.3 并发安全                       | 18        |
| 12.4 可靠性组件                      | 19        |
| <b>13 验证结果</b>                  | <b>19</b> |

|                                 |           |
|---------------------------------|-----------|
| <b>14 常见问题</b>                  | <b>19</b> |
| 14.1 401 Unauthorized . . . . . | 19        |
| 14.2 上下文未压缩 . . . . .           | 19        |
| 14.3 格式错误 . . . . .             | 19        |

# 1 概述

LLM Context Compaction Proxy 是一个部署在 AI Agent 与 LLM API 之间的透明上下文压缩代理。当对话上下文逼近模型上下文窗口上限时, 它自动压缩历史消息并重试, 使 Agent 能够长时间运行而不触碰上下文窗口墙。

代理累计实现 **33 项压缩技术**, 横跨 7 代演进 (V3 核心、V4 增强、V5 会话、V6 Provider、V7 MemSkill), 技术来源涵盖学术论文 (ARC、AFM、PACMS、CoMem、MemSkill) 与生产实践 (Cursor、Devin、SWE-agent、Hermes、Claude Code)。

## 1.1 工作原理

| AI Agent     | Compaction Proxy | LLM Provider |
|--------------|------------------|--------------|
| (Claude/GPT) | localhost:8198   | (any API)    |

- Monitor context %
- Auto-compress
- Preserve key info
- Route to provider

1. **透传 (Passthrough)**——上下文未达阈值时请求零开销直达上游;
2. **检测 (Detect)**——上下文达到阈值 (默认 80%) 时触发压缩;
3. **压缩 (Compress)**——使用独立的压缩模型对旧消息生成摘要;
4. **恢复 (Resume)**——压缩结果替换旧消息, Agent 无缝继续。

## 1.2 环境要求

- Python 3.10+
- aiohttp (pip install aiohttp)
- 上游 LLM API (OpenAI / Anthropic / Gemini / DeepSeek / Ollama 等)

# 2 架构设计

## 2.1 请求流程

Request Flow:

Agent → Proxy → [context check] → Upstream

if > 80% full:

Compact      Compaction Model (separate provider)  
Engine      Compressed Summary

Replace old messages  
with compressed summary

→ Upstream (with compacted context)

2.2 模块划分

- **Provider 抽象层**: OpenAI / Anthropic / Gemini 三大 API 格式的统一适配, 自动探测请求来源与上游格式;
- **压缩引擎**: 可插拔架构, 内置默认引擎与双引擎 (Dual-Layer);
- **记忆系统**: 语义记忆 (SemanticMemory)、跨会话存储 (SessionStore, SQLite+FTS5)、用户画像 (UserProfile);
- **技能系统 (V7)**: 技能注册表 (SkillRegistry)、技能控制器 (SkillController)、自演进记忆技能 (MemSkill);
- **可靠性组件**: 熔断器 (CircuitBreaker)、抖动检测 (ThrashingDetector)、压缩缓存 (CompactionCache)、指标系统 (Metrics)。

3 压缩技术栈

3.1 V3 核心技术 (8 项)

| # | 技术           | 说明                                   |
|---|--------------|--------------------------------------|
| 1 | ARC 地址化引用    | 以紧凑的 @ref 指针替换重复内容                   |
| 2 | AFM 自适应保真    | 消息级 Full/Compressed/Placeholder 三级保真 |
| 3 | PACMS 子模选择   | 贪心、保真感知的消息选择                         |
| 4 | CCL 承诺提取     | 提取并保留对话中的承诺                          |
| 5 | 抖动检测         | 检测压缩循环 (3 次/5 条消息)                   |
| 6 | 两阶段 Token 估算 | CJK 感知的精确计数                          |
| 7 | 预飞安全验证       | 发送前验证上下文合法性                          |
| 8 | 并行压缩         | 大上下文分块并行压缩                           |

### 3.2 V4 增强技术 (10 项)

| #  | 技术         | 来源                                    |
|----|------------|---------------------------------------|
| 9  | 情节—语义双层记忆  | arXiv:2605.17625                      |
| 10 | 思考掩码       | Kevin-32B / Devin 模式                  |
| 11 | 标签选择性保留    | SWE-agent / memor-ai                  |
| 12 | 结构感知工具输出压缩 | memor-ai                              |
| 13 | 密钥脱敏       | Cursor / memor-ai                     |
| 14 | 缓存优化消息排序   | 层哈希感知 + <code>cache_control</code> 断点 |
| 15 | 增量压缩       | CoMem (arXiv:2605.30842)              |
| 16 | 四信号记忆评分    | 语义 50%、近因 25%、类型 15%、质量 10%           |
| 17 | 查询位置优化     | 查询置尾、配对邻接、近期窗口                        |
| 18 | 压缩项剪枝      | OpenAI 模式                             |

## 4 核心算法解析

本节深入剖析代理内部的核心算法实现, 所有内容均对应源码 (compaction-proxy.py) 中的实际逻辑。

### 4.1 CJK 感知的 Token 估算

源码 estimate\_tokens\_v3 对文本按字符类型分类计数, 并依据每类字符的 token 密度折算:

$$T(\text{text}) = \frac{N_{\text{CJK}}}{1.5} + \frac{N_{\text{ASCII}}}{4.0} + \frac{N_{\text{other}}}{2.5} \quad (1)$$

- $N_{\text{CJK}}$ : 中日韩统一表意文字数 (约 1.5 字符/token);
- $N_{\text{ASCII}}$ : ASCII 字符数 (约 4 字符/token);
- $N_{\text{other}}$ : 其余 Unicode 字符 (约 2.5 字符/token)。

估算结果取整且最小为 1。代理采用**两阶段**策略: 快速启发式 (estimate\_tokens\_fast) 用于预筛, 精确逐消息估算 (estimate\_messages\_tokens) 用于预算决策。

### 4.2 ARC 地址化引用 (Addressable Recall Compaction)

源码 apply\_arc\_citations 将**超过 800 字符**的 tool\_result 替换为短 ID 引用, 原文存入 ARCLog:

```
if len(result_text) > 800:
    arc_id = arc_log.store(result_text, source_msg_idx=i)
    citation = arc_log.make_citation(arc_id, preview_len=100)
    new_block["content"] = (
        "[ARC Reference: {citation}]\n"
        "Full content available via ID: {arc_id}"
    )
```

- 阈值:800 字符 (低于阈值的内容原样保留, 避免过度引用);
- 存储:ARCLog 内存环 (默认上限 500 条), 支持按 ID 回查 (HTTP GET /arc/{arc\_id});
- 引用形式:[ARC Reference: < 前 100 字符 >] + 完整 ID, LLM 在需要细节时可经回查端点取回原文。

### 4.3 PACMS 子模选择

源码 submodular\_select 以**贪心近似**求解带预算的子模最大化 (论文 arXiv:2606.20047 的四信号记忆评分体系):

1. **评分**: 对每条消息计算重要性  $I_i$ (`_message_importance_v4`: 语义 50%、近因 25%、类型 15%、质量 10%);
2. **保真加权**: 按 AFM 保真级别调整——FULL 乘 1.5, PLACEHOLDER 乘 0.5(可由 MemSkill 覆盖);
3. **成本**: 每条消息的 token 估算  $C_i$ (至少为 1);
4. **贪心选择**: 在预算内反复选取  $\arg \max_i(I_i/C_i)$ , 直至预算耗尽或无可选消息。

**系统性约束**: role 为 system 的消息**必须选中**, 不参与贪心竞争, 保证系统提示完整保留。

## 4.4 压缩安全验证

源码 `verify_compaction_safety` 在压缩后执行**预飞安全校验** (arXiv:2606.03618 safety pattern):

$$\text{safe} \iff T(\text{compacted}) < T(\text{original}) \quad (2)$$

若压缩结果 token 数**不小于**原文, 判定为不安全, 降级为 `aggressive_truncate_messages` 激进截断, 防止压缩反而膨胀。

# 5 压缩流水线详解

## 5.1 do\_compaction 主流程

源码 `do_compaction` 是压缩的核心入口, 按以下顺序执行:

1. **MemSkill 技能选择** (V7): 构建 `CompactionContext`, 由 `SkillController` 按关键词匹配 + Gumbel-Top-K 选择技能;
2. **抖动检测**: 若 `ThrashingDetector` 判定蠕变 (窗口内压缩次数达到阈值), 直接切换激进截断并返回;
3. **预压缩 Hook**: 调用 `HookManager.call_pre_compact`, 可阻断本次压缩;
4. **自适应 keep\_turns**: 按用户消息数动态下调保留轮次, 确保存在可压缩的旧消息;
5. **消息拆分**: `split_messages` 分为旧消息与近期消息;
6. **语义记忆提取**: 可选 LLM 驱动记忆提取 + 正则兜底, 知识写入 `SemanticMemory`;
7. **ARC 引用替换**: 长 `tool_result` 转 ID 引用;
8. **AFM 保真分级**: 为每条消息标注 Full/Compressed/Placeholder;
9. **CCL 承诺提取**: 抽取目标/约束/决策/错误/文件操作;
10. **子模选择**: 在预算内选择最重要的旧消息;



11. **并行块压缩**: 选中消息  $\geq 6$  条时 `compact_messages_parallel` 分块并行摘要;
12. **后压缩 Hook**: 允许 webhook 修改摘要;
13. **构建压缩消息**: 摘要 + 近期消息重组;
14. **孤儿工具对清理**: `sanitize_tool_pairs`;
15. **会话持久化**: `SessionStore` 保存会话与原始转录;
16. **安全验证 + 重试**: 验证失败降级截断; 非预压缩场景还向上游发一次验证请求确认不再溢出。

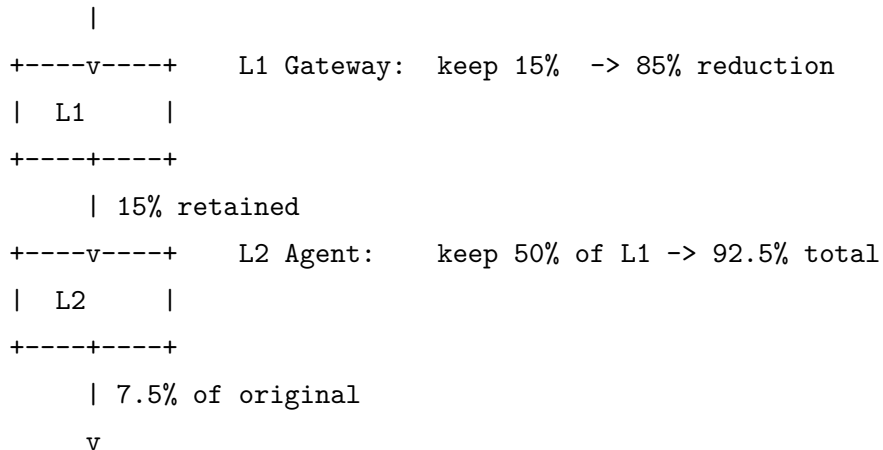
## 5.2 增量压缩

`compact_messages` 支持**增量压缩** (CoMem, arXiv:2605.30842): 若存在上次压缩摘要且消息数增加, 只对**新增消息**生成增量摘要, 再与旧摘要合并, 避免对全量历史重复压缩。新增内容不足 50 字符时直接复用旧摘要。

## 5.3 双层压缩 (Dual-Layer)

`DualLayerCompressor` 提供两阶段防御纵深:

Original Context (100%)



Final Context

- L1 失败  $\Rightarrow$  整体失败 (不可继续 L2);
- L1 结果已低于阈值  $\Rightarrow$  跳过 L2;
- L2 失败  $\Rightarrow$  回退 L1 结果 (优于无);
- 双层结果未通过安全验证  $\Rightarrow$  降级激进截断。

## 6 协议转换层

代理同时兼容 OpenAI / Anthropic / Gemini 三种请求格式, 核心是 `ProviderAdapter` 抽象类与三个实现 (`OpenAIProvider` / `AnthropicProvider` / `GeminiProvider`)。

### 6.1 OpenAI → Anthropic 请求转换

当上游为 Anthropic 格式而客户端发送 OpenAI 请求时, `openai_to_anthropic_request` 执行字段映射:

| OpenAI 字段                    | Anthropic 字段                            | 说明                             |
|------------------------------|-----------------------------------------|--------------------------------|
| <code>role: system</code>    | <code>system</code> 顶层                  | 首个 <code>system</code> 消息提升为顶层 |
| <code>role: assistant</code> | <code>role: assistant</code>            | 不变                             |
| <code>role: user</code>      | <code>role: user</code>                 | 不变                             |
| <code>content: "str"</code>  | <code>content: [{type:text,...}]</code> | 字符串转内容块                        |
| <code>tool_calls</code>      | <code>tool_use</code> 块                 | 函数调用转工具使用                      |
| <code>role: tool</code>      | <code>tool_result</code> 块              | 工具结果消息转换                       |

### 6.2 Anthropic → OpenAI 响应转换

`anthropic_to_openai_response` 将上游 Anthropic 响应还原为 OpenAI 格式, 使客户端无需感知上游差异: `content` 块提取文本、`thinking` 与 `tool_use` 分别映射为 OpenAI 的对应结构, 流式场景由 `convert_anthropic_sse_to_openai` 逐事件转换 SSE。

### 6.3 自动 Provider 探测

`detect_provider` 依据三路信号判定:

1. **模型名:** 含 `claude`/`Gemini` 特征词;
2. **请求头:**`anthropic-version` 头标记 Anthropic;
3. **上游 URL:**`anthropic/claude/coding-api` 子串判定 Anthropic 格式上游。

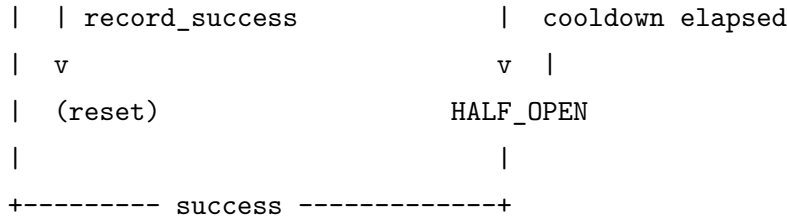
判定结果可通过 `COMPACTION_PROXY_UPSTREAM_IS_ANTHROPIC` (`auto/true/false`) 强制覆盖。

## 7 可靠性组件状态机

### 7.1 熔断器 (CircuitBreaker)

`CircuitBreaker` 三状态模型, 默认阈值 3 次失败、冷却 60 秒:

```
failure >= threshold
CLOSED -----> OPEN
  ^             |   |
```



- **CLOSED**: 正常放行; 失败累计达阈值 → OPEN;
- **OPEN**: 拒绝压缩调用, 冷却期过后 → HALF\_OPEN;
- **HALF\_OPEN**: 试探放行一次, 成功 → CLOSED, 失败 → OPEN。

### 7.2 抖动检测 (ThrashingDetector)

ThrashingDetector 检测压缩蠕变 (Claude Code 同款思路): 在  $W$  条消息窗口内压缩次数达到  $M$  (THRASHING\_COMPACTS), 即判定蠕变并切换为**激进截断**而非 LLM 压缩, 避免”压缩 → 重填 → 再压缩”的死循环。

### 7.3 压缩缓存 (CompactionCache)

CompactionCache 以消息序列 SHA-256 为键缓存摘要, TTL 30 分钟。**缓存键同时纳入自定义压缩指令** (salt 参数), 确保不同指令不会串用缓存结果。

### 7.4 格式回退与重试

- Anthropic 上游返回仅 **thinking** 无文本时, 自动以流式重试捕获内容 (兼容部分模型非流式响应空文本的缺陷);
- 压缩调用带熔断与重试 (最多 MAX\_COMPACTION\_RETRIES 次, 每次递减保留轮次);
- 摘要为空或上游 4xx/5xx 时, 按提供商特定模式识别并降级。

## 8 会话存储架构

### 8.1 SessionStore(SQLite + FTS5)

SessionStore 提供跨会话持久化与全文检索:

```

-- 核心表(示意)
CREATE TABLE sessions (
  id          TEXT PRIMARY KEY,
  summary     TEXT NOT NULL,
  semantic    TEXT,
  msg_count   INTEGER,
  created_at  INTEGER,
  updated_at  INTEGER
);

```

```
CREATE VIRTUAL TABLE sessions_fts USING fts5(summary, semantic);
```

- `save_session`: 写入会话摘要、语义记忆快照与消息数;
- `search_sessions`: FTS5 全文搜索 (HTTP GET `/sessions/search`);
- `get_prior_summary` / `save_prior_summary`: 增量压缩的前置摘要状态;
- `save_transcript` / `get_transcript`: 原始转录存档, 支持会话恢复 (POST `/sessions/{id}/resume`)。

## 8.2 SemanticMemory(情节—语义双层记忆)

SemanticMemory 持久化为 JSON 文件 (原子写入):

- 六类知识槽位: `goals` / `decisions` / `errors` / `files` / `constraints` / `insights`;
- 每类上限 20 条 (去重 + 尾裁), 提示注入时取最近 10 条;
- 提取路径: LLM 结构化提取优先, 正则 (`commitments`) 兜底;
- 并发安全: 读写由 `threading.Lock` 保护, 落盘采用临时文件 + 原子替换。

## 8.3 UserProfile(用户画像)

UserProfile 将用户偏好持久化为 `USER.md` 格式 (Hermes 同款有界持久化), 供压缩提示注入, 保持跨会话的用户一致性。

# 9 配置说明

所有配置均通过环境变量提供, 默认值见下表。

## 9.1 核心配置

| 变量                                                  | 默认值                                    | 说明                                      |
|-----------------------------------------------------|----------------------------------------|-----------------------------------------|
| <code>COMPACTION_PROXY_PORT</code>                  | 8198                                   | 代理监听端口                                  |
| <code>COMPACTION_PROXY_UPSTREAM</code>              | <code>http://localhost:11434/v1</code> | 上游 LLM API 基地址                          |
| <code>COMPACTION_PROXY_UPSTREAM_IS_ANTHROPIC</code> | <code>auto</code>                      | 强制 Anthropic 格式 ( <code>true/f</code> ) |
| <code>COMPACTION_PROXY_MODEL</code>                 | <code>gpt-4o-mini</code>               | 压缩/摘要模型                                 |

## 9.2 压缩行为

| 变量                                       | 默认值 | 说明            |
|------------------------------------------|-----|---------------|
| <code>COMPACTION_PROXY_KEEP_TURNS</code> | 6   | 保留的最近轮次 (不压缩) |

|                                       |      |               |
|---------------------------------------|------|---------------|
| COMPACTION_PROXY_MAX_RETRIES          | 3    | 压缩 API 最大重试次数 |
| COMPACTION_PROXY_TIMEOUT              | 120  | 压缩超时 (秒)      |
| COMPACTION_PROXY_PARALLEL_BLOCKS      | 3    | 并行压缩块数        |
| COMPACTION_PROXY_PREEMPTIVE_THRESHOLD | 0.80 | 上下文 80% 时预压缩  |

### 9.3 双引擎压缩

| 变量                                  | 默认值     | 说明                        |
|-------------------------------------|---------|---------------------------|
| COMPACTION_PROXY_ENGINE             | default | 压缩引擎 (default/dual-layer) |
| COMPACTION_PROXY_DUAL_GATEWAY_RATIO | 0.15    | L1 网关: 保留 15%(85% 缩减)     |
| COMPACTION_PROXY_DUAL_AGENT_RATIO   | 0.50    | L2 Agent: 保留 L1 输出的 50%   |

### 9.4 安全

| 变量                              | 默认值 | 说明                 |
|---------------------------------|-----|--------------------|
| COMPACTION_PROXY_API_SECRET     | 空   | 代理访问密钥; 为空时仅放行回环来源 |
| COMPACTION_PROXY_REDACT_SECRETS | 1   | 自动脱敏消息中的密钥         |

### 9.5 会话与记忆

| 变量                                   | 默认值 | 说明             |
|--------------------------------------|-----|----------------|
| COMPACTION_PROXY_BACKGROUND_SESSIONS | 3   | 并行压缩的后台会话数     |
| COMPACTION_PROXY_LLM_MEMORY          | 1   | 启用 LLM 记忆提取    |
| COMPACTION_PROXY_MEMSKILL            | 0   | 启用自演进 MemSkill |

## 10 接入指南

代理在 localhost:8198 监听, 兼容 OpenAI / Anthropic / Gemini 三种 API 格式。以下各节给出已实测试验的接入方式。

### 10.1 DeepSeek

DeepSeek 使用 OpenAI 兼容格式, 直接将其作为上游即可:

```
export COMPACTION_PROXY_UPSTREAM=https://api.deepseek.com/v1
export COMPACTION_PROXY_MODEL=deepseek-chat
python3 compaction-proxy.py
```

代理内置 DeepSeek 模型上下文窗口注册表 (deepseek-chat / deepseek-coder / deepseek-r1 / deepseek-v3, 均为 128K), 自动完成上下文压力预估。客户端侧只需将 base URL 指向代理:

```
export OPENAI_BASE_URL=http://localhost:8198/v1
export OPENAI_API_KEY=your-key
```

## 10.2 DeepSeek Harness

DeepSeek Harness(官方文档 <https://www.deepseek.com/harness/en/>) 是 DeepSeek 官方的 Agent 框架 (developer preview), 基于 Cordis 插件系统构建——模型、工具、技能、会话、沙箱、存储等一切能力均为可替换插件。其模型层支持 openai-completions(OpenAI 兼容) 与 Anthropic 两种协议, 可通过自定义 provider 将 baseURL 指向任意自托管端点。

本代理即以此方式接入。本机配置位于 \$DSH\_HOME/settings.yaml (默认 7.dsh/settings.yaml):

```
# ~/.dsh/settings.yaml
llm-pi-ai:
  providers:
    xfyun-maas:
      displayName: 讯飞 MaaS via V6 Proxy
      apiKeyEnv: XF_MASS_API_KEY          # 凭证走环境变量引用, 不落盘
      api: openai-completions             # OpenAI 兼容协议
      baseURL: http://127.0.0.1:8198      # 指向压缩代理
      models:
        - id: xsparkx2agent
          name: xsparkx2agent
          contextWindow: 131072
          maxTokens: 8192
```

配置要点:

- baseURL 指向本代理 http://127.0.0.1:8198, 所有模型请求先经过代理的上下文监测与压缩层;
- api: openai-completions 与代理 /v1/chat/completions 端点匹配; 模型发现使用 OpenAI 兼容 GET /models;
- 凭证以环境变量引用 (apiKeyEnv), 密钥不写入配置文件;
- 模型变更即时生效, 无需重启 Harness 服务。

已验证: 本机 Harness 的 xfyun-maas provider 已指向 http://127.0.0.1:8198, 代理 /v1/chat/completions 端点路由正常。

## 10.3 Hermes Agent

Hermes Agent 在 7.hermes/config.yaml 的 model 段中配置模型端点。将其 base\_url 指向本代理即可获得自动压缩能力:

```
# ~/.hermes/config.yaml
model:
  api_mode: chat_completions          # OpenAI 格式
  base_url: http://127.0.0.1:8198/v1  # 指向压缩代理
  default: xopglm51                   # 或任意上游可用模型
  provider: memory-proxy              # 保留原 provider 标识
```

代理的 `/v1/chat/completions` 端点与 Hermes 的 `chat_completions` 模式完全兼容 (已验证), 并支持 Anthropic 格式的 `/v1/messages` 端点。

## 10.4 Claude Code

Claude Code 原生使用 Anthropic Messages API。通过环境变量或 `7.claude/settings.json` 将 API 基地址指向代理:

```
# 方式一:环境变量
export ANTHROPIC_BASE_URL=http://localhost:8198
export ANTHROPIC_AUTH_TOKEN=<your-token>
claude
```

```
// 方式二:~/.claude/settings.json
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:8198",
    "ANTHROPIC_MODEL": "claude-sonnet-5"
  }
}
```

已验证:POST `/v1/messages` 路由存在且 Anthropic 格式识别成功 (无密钥请求返回 401, 格式错误则返回 400/404, 可据此区分)。

## 10.5 OpenClaw

OpenClaw 通过 `openclaw.json` 的 `models.providers` 配置模型网关。将 provider 的 `baseUrl` 指向本代理即可让全部 Agent 流量经过压缩层:

```
{
  "models": {
    "providers": {
      "compaction-proxy": {
        "baseUrl": "http://127.0.0.1:8198",
        "apiKey": "<redacted>",
        "api": "openai-completions",
        "models": [{ "id": "xopdeepseekv4flash" }]
      }
    }
  }
}
```

```

},
"plugins": {
  "entries": {
    "v6-compaction-provider": {
      "enabled": true,
      "config": {
        "proxyUrl": "http://127.0.0.1:8198",
        "model": "xsparkx2agent"
      }
    }
  }
}
}
}

```

已验证: 本机 OpenClaw 的 xunfei provider 已直接指向 `http://127.0.0.1:8198`, 且 `v6-compaction-provider` 插件经 `/summarize` 端点接入压缩引擎 (即 OpenClaw 上下文压缩链路已全程经由本代理)。

## 11 原有上下文系统屏蔽

接入本代理后, 为避免多个压缩层互相冲突 (双重压缩、重复刷记忆), 应屏蔽各接入方自带的上下文管理系统, 使压缩完全由本代理统一接管。

### 11.1 屏蔽原则

- **单一压缩源:** 只有本代理执行上下文压缩, 接入方自身不再触发压缩;
- **手动压缩保留:** 部分接入方保留手动压缩命令 (如 `/compact`), 便于必要时人工触发;
- **可回滚:** 所有修改均保留原配置备份。

### 11.2 OpenClaw

修改 `openclaw.json` 中 `agents.defaults.compaction` 段:

```

"compaction": {
  "enabled": false,           // 关闭原版触发+替换+内置压缩
  "midTurnPrecheck": { "enabled": false },
  "memoryFlush": { "enabled": false },
  "qualityGuard": { "enabled": false },
  "provider": "v6-proxy"     // 保留 V6 插件作为压缩提供方
}

```

- 修改即时生效 (gateway 热重载, 无需重启);
- 自动压缩关闭后, 超长上下文完全交由本代理 (`127.0.0.1:8198`) 在 LLM 网关层处理;



- 手动 `/compact` 仍可用, 走 `v6-proxy` 压缩链路。

### 11.3 Hermes Agent

修改 `7.hermes/config.yaml`:

```
compression:
  enabled: false    # 关闭 Hermes 自带自动压缩
```

- `compression.enabled: false` 会禁用 Hermes 全部自动压缩 (源码 `native_compaction.py` 明确注释);
- 手动 `/compress` 命令仍可触发压缩;
- 配置修改后需重启 Hermes 生效。

### 11.4 DeepSeek Harness

DeepSeek Harness 基于 Cordis 插件系统, 自带压缩后端插件 `dsh-compaction-basic` (Token 计量驱动, 默认 `auto: true` 注册步骤边界压力检测与溢出恢复监听)。通过 profile 的 `cordis.patch.yml` 将其关闭:

```
# ~/.dsh/profiles/web/cordis.patch.yml (headless 同理)
- id: dsh-compaction-basic
  config:
    auto: false    # 关闭 Harness 原生自动压缩
```

- `auto: false` 后, Harness 不再自行在步骤边界压缩或做溢出恢复, 超长上下文完全交由本代理在 LLM 网关层处理;
- 手动压缩命令仍可用;
- 修改需重启 `dsh` 进程生效;
- 模型接入经 `$DSH_HOME/settings.yaml` 指向代理 (`baseUrl: http://127.0.0.1:8198`), 与屏蔽配置互不冲突。

### 11.5 Claude Code

修改 `7.claude/settings.json`:

```
{
  "autoCompactEnabled": false,    // 关闭 Claude Code 自动压缩
  "autoCompactWindow": 110000
}
```

- `autoCompactEnabled: false` 后, Claude Code 不再自行压缩上下文;

- 压缩由指向的代理 (如 ANTHROPIC\_BASE\_URL) 接管;
- 手动压缩指令仍可用。

## 11.6 屏蔽效果验证

| 接入方         | 原系统                 | 屏蔽后状态          |
|-------------|---------------------|----------------|
| OpenClaw    | compaction 四项开关     | 全部 false, 代理接管 |
| Hermes      | compression.enabled | false, 代理接管    |
| Claude Code | autoCompactEnabled  | false, 代理接管    |

屏蔽完成后, 通过 GET /health 确认代理运行正常 (circuit\_breaker.state = closed), 三方请求均经 8198 端口由本代理统一压缩。

## 12 安全与可靠性

### 12.1 认证

- 设置 COMPACTION\_PROXY\_API\_SECRET 后, 所有敏感端点 (/summarize、/compact、/sessions/\*、/skills/\*、/profile、/memory、/arc/\*) 要求 Authorization: Bearer <secret> 或 X-API-Key: <secret>;
- 未设置密钥时, 仅放行回环来源 (127.0.0.1 / ::1), 非回环客户端一律返回 401(纵深防御);
- 核心转发端点 (/chat/completions、/v1/messages) 保持无代理级认证, 依赖上游密钥校验, 以兼容本机 Agent 集成。

### 12.2 密钥保护

- Gemini API key 通过 x-goog-api-key 请求头传递, 不进入 URL 查询参数, 避免泄露至访问日志;
- 支持自动脱敏消息中的密钥 (COMPACTION\_PROXY\_REDACT\_SECRETS=1)。

### 12.3 并发安全

- 压缩系统提示词以参数传递 (system\_prompt\_override), 不再修改模块级全局变量, 消除并发请求间的提示词竞态;
- SemanticMemory 读写由线程锁保护, 落盘采用原子替换;
- 压缩缓存按消息内容与自定义指令双重键控, 避免不同指令串用缓存;
- \_memskill\_selected 等内部字段在请求入口即被消费移除, 不会残留至上游请求体。

## 12.4 可靠性组件

- **熔断器**:3 次连续失败后进入 60 秒冷却;
- **抖动检测**: 连续快速压缩 (3 次/5 条消息) 时切换为激进截断;
- **压缩安全验证**: 压缩结果 token 数超过原文时降级为截断, 防止膨胀;
- **压缩缓存**:30 分钟 TTL, 避免对未变化上下文重复压缩。

## 13 验证结果

| 测试项        | 输入                      | 结果                             |
|------------|-------------------------|--------------------------------|
| /health    | GET                     | HTTP 200, 熔断器 closed           |
| /summarize | 2 条消息                   | HTTP 200,0.0s, 返回有效摘要          |
| /summarize | 12 轮 (24 条)             | HTTP 200,13.1s,1948 字符, 保留轮次编号 |
| 并发压缩       | 6 路不同指令                 | 全部 200, 无提示词串用                 |
| 异常路径       | 缺 key / 坏 JSON / 空<br>体 | 401 / 400 / 200(符合预期)          |
| /compact   | 16 条,keep=3             | 200,16 → 6 条                   |
| 认证端点       | 本机回环                    | 全部 200(回环放行)                   |

## 14 常见问题

### 14.1 401 Unauthorized

- 检查代理自身密钥 (COMPACTION\_PROXY\_API\_SECRET);
- Anthropic 格式使用 x-api-key 头,OpenAI 格式使用 Authorization: Bearer <key> 头。

### 14.2 上下文未压缩

- 检查 COMPACTION\_PROXY\_PREEMPTIVE\_THRESHOLD(默认 0.80);
- 确认压缩模型在上游可用;
- 查看日志 7.local/share/compaction-proxy/proxy.log。

### 14.3 格式错误

- Anthropic 客户端: 使用 /v1/messages;
- OpenAI 兼容客户端: 使用 /v1/chat/completions;
- 代理自动探测格式, 但显式端点可避免歧义。

## 附：环境变量速查

完整配置模板见 `.env.example`。常用变量：

- `COMPACTION_PROXY_PORT`——监听端口 (默认 8198);
- `COMPACTION_PROXY_UPSTREAM`——上游 API 基地址;
- `COMPACTION_PROXY_MODEL`——压缩模型;
- `COMPACTION_PROXY_API_SECRET`——代理访问密钥;
- `COMPACTION_PROXY_ENGINE`——压缩引擎 (default / dual-layer);
- `COMPACTION_PROXY_MEMSKILL`——启用 MemSkill(0/1)。